ASSIGNMENT SET 6

1. Simulate the undo and redo option in word packages using a doubly linked list.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct node {
   char text[100];
   struct node *prev, *next;
};

struct node *head = NULL, *cur = NULL;

void add(char s[]) {
   struct node *n = malloc(sizeof(struct node));
   strcpy(n->text, s);
   n->prev = cur;
   n->next = NULL;

   if (cur != NULL) {
      struct node *p = cur->next;
      while (p != NULL) {
         struct node *t = p;
         p = p->next;
         free(t);
      }
      cur->next = n;
   } else {
      head = n;
   }

   cur = n;
}

void undo() {
   if (cur != NULL) cur = cur->prev;
   else printf("Nothing to undo\n");
}
```

```
void redo() {
   if (cur != NULL && cur->next != NULL)
      cur = cur->next;
   else printf("Nothing to redo\n");
}

void display() {
   if (cur == NULL) printf("Document is empty\n");
   else printf("Current text: %s\n", cur->text);
}

int main() {
   int ch;
   char s[100];

   while (1) {
      printf("\n1.Add 2.Undo 3.Redo 4.Display 5.Exit\n");
      printf("Enter choice: ");
      scanf("%d", &ch);

      switch (ch) {
         case 1:
            printf("Enter text: ");
            scanf(" %99[^\n]", s);
            add(s);
            break;
         case 2: undo(); break;
         case 3: redo(); break;
         case 4: display(); break;
         case 5: return 0;
         default: printf("Invalid choice\n");
      }
   }
}
```

OUTPUT:

```
1.Add 2.Undo 3.Redo 4.Display 5.Exit
Enter choice: 1
Enter text: hello

1.Add 2.Undo 3.Redo 4.Display 5.Exit
Enter choice: 1
Enter text: bye

1.Add 2.Undo 3.Redo 4.Display 5.Exit
Enter choice: 2

1.Add 2.Undo 3.Redo 4.Display 5.Exit
Enter choice: 3

1.Add 2.Undo 3.Redo 4.Display 5.Exit
Enter choice: 4
Current text: bye

1.Add 2.Undo 3.Redo 4.Display 5.Exit
Enter choice: 2

1.Add 2.Undo 3.Redo 4.Display 5.Exit
Enter choice: 4
Current text: hello

1.Add 2.Undo 3.Redo 4.Display 5.Exit
Enter choice: 5
PS C:\Users\R Cristina Jane\OneDrive\Desktop\D
```

2. Given two sorted lists L1 and L2, write a procedure to compute L1 ∩ L2 and L1 ∪ L2 using basic list operations.

```
#include <stdio.h>

int main() {
   int a[50], b[50], u[100], in[50];
   int n, m, i = 0, j = 0, k = 0, p = 0;

   printf("Enter size of L1: ");
   scanf("%d", &n);
   printf("Enter sorted elements of L1: ");
   for (int x = 0; x < n; x++) scanf("%d", &a[x]);

   printf("Enter size of L2: ");
   scanf("%d", &m);
   printf("Enter sorted elements of L2: ");
```

```
    for (int x = 0; x < m; x++) scanf("%d", &b[x]);

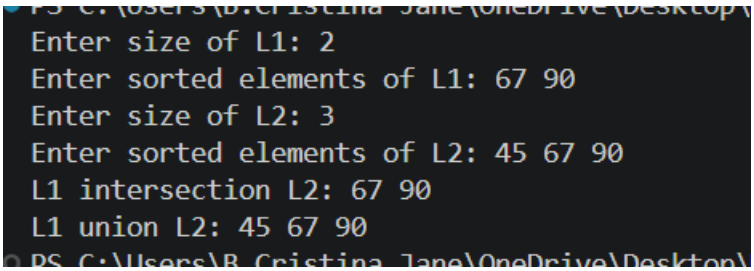
    while (i < n && j < m) {
        if (a[i] < b[j]) u[k++] = a[i++];
        else if (a[i] > b[j]) u[k++] = b[j++];
        else {
            u[k++] = a[i];
            in[p++] = a[i];
            i++;
            j++;
        }
    }

    while (i < n) u[k++] = a[i++];
    while (j < m) u[k++] = b[j++];

    printf("L1 intersection L2: ");
    for (i = 0; i < p; i++) printf("%d ", in[i]);

    printf("\nL1 union L2: ");
    for (i = 0; i < k; i++) {
        if (i == 0 || u[i] != u[i - 1])
            printf("%d ", u[i]);
    }
    printf("\n");
    return 0;
}
```

OUTPUT:



3. Swap two elements by adjusting only the pointers (and not the data) using doubly linked list.

```c
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *prev, *next;
};

struct node *head = NULL;

void insert(int x) {
   struct node *n = malloc(sizeof(struct node));
   n->data = x;
   n->next = NULL;
   n->prev = NULL;

   if (head == NULL) head = n;
   else {
      struct node *t = head;
      while (t->next != NULL) t = t->next;
      t->next = n;
      n->prev = t;
   }
}

void display() {
   struct node *t = head;
   while (t != NULL) {
      printf("%d ", t->data);
      t = t->next;
   }
   printf("\n");
}

/* Swap nodes by changing links, not data */
void swapNodes(int x, int y) {
   struct node *a = head, *b = head;

   while (a && a->data != x) a = a->next;
   while (b && b->data != y) b = b->next;
```

```
    if (!a || !b || a == b) return;

    if (a->next == b) {
        struct node *p = a->prev, *n = b->next;
        if (p) p->next = b;
        else head = b;
        if (n) n->prev = a;
        b->prev = p; b->next = a;
        a->prev = b; a->next = n;
    } else if (b->next == a) {
        swapNodes(y, x);
        return;
    } else {
        struct node *ap = a->prev, *an = a->next;
        struct node *bp = b->prev, *bn = b->next;

        if (ap) ap->next = b;
        else head = b;
        if (an) an->prev = b;
        if (bp) bp->next = a;
        else head = a;
        if (bn) bn->prev = a;

        a->prev = bp; a->next = bn;
        b->prev = ap; b->next = an;
    }
}

int main() {
    int n, x, y, v;

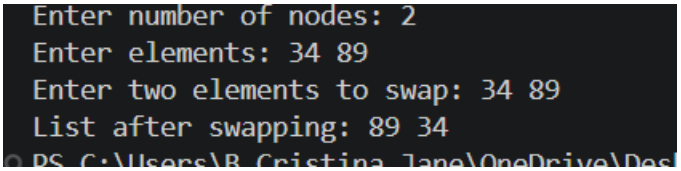
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &v);
        insert(v);
    }

    printf("Enter two elements to swap: ");
    scanf("%d%d", &x, &y);
```

```
    swapNodes(x, y);

    printf("List after swapping: ");
    display();
    return 0;
}
```

OUTPUT:



4. Write an array implementation of self adjusting lists. A self adjusting list is like a regular list, except that all insertions are performed at the front and when an element is accessed by a find, it is moved to the front of the list, without changing the relative order of the other elements.

```
#include <stdio.h>

int main() {
    int a[100], n, choice, x, i, pos, temp;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    while (1) {
        printf("\n1. Insert at front");
        printf("\n2. Find element");
        printf("\n3. Display list");
        printf("\n4. Exit");
        printf("\nEnter choice: ");
        scanf("%d", &choice);
```

```
switch (choice) {
   case 1:
      printf("Enter element: ");
      scanf("%d", &x);

      for (i = n; i > 0; i--) {
         a[i] = a[i - 1];
      }

      a[0] = x;
      n++;
      break;

   case 2:
      printf("Enter element to find: ");
      scanf("%d", &x);

      for (pos = 0; pos < n; pos++) {
         if (a[pos] == x)
            break;
      }

      if (pos == n) {
         printf("Element not found\n");
      } else {
         temp = a[pos];

         for (i = pos; i > 0; i--) {
            a[i] = a[i - 1];
         }

         a[0] = temp;
         printf("Element found and moved to front\n");
      }
      break;

   case 3:
      printf("List: ");
      for (i = 0; i < n; i++) {
         printf("%d ", a[i]);
      }
```

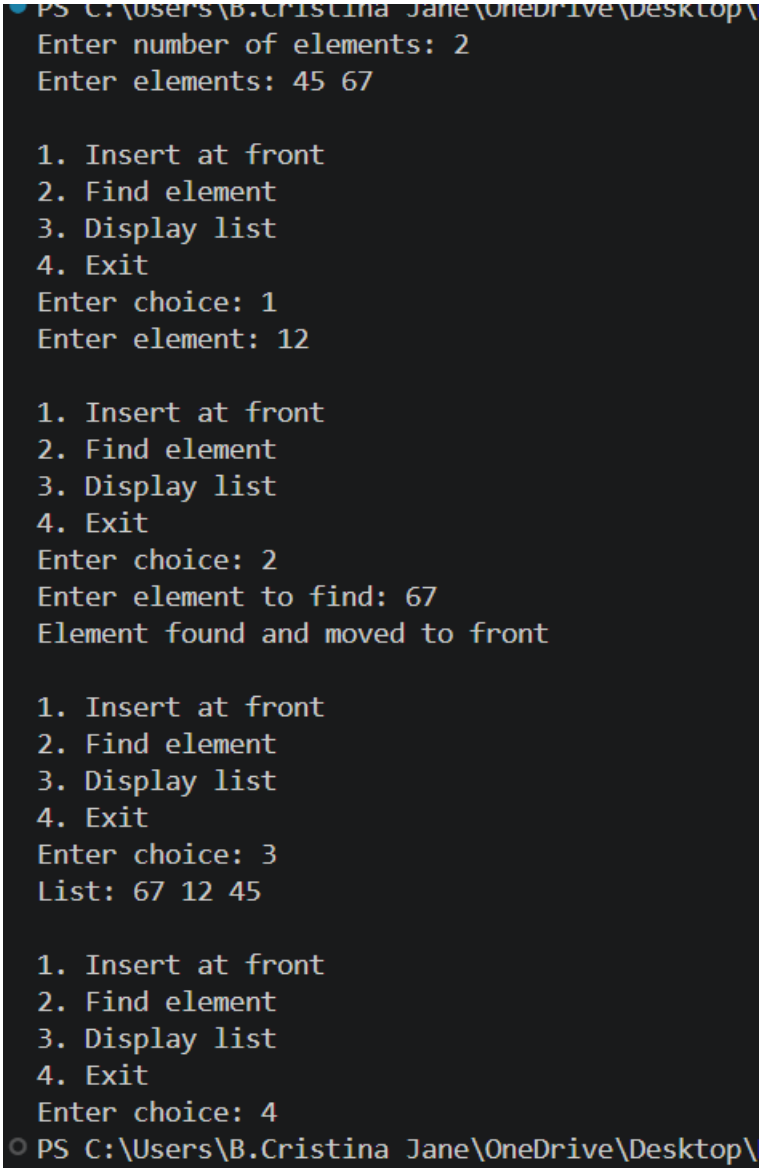
```
            printf("\n");
            break;

        case 4:
            return 0;

        default:
            printf("Invalid choice\n");
      }
   }

   return 0;
}
```

OUTPUT:



4. Write a C program to reverse a given doubly linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *prev, *next;
};

struct node *head = NULL;

void insert(int x) {
   struct node *n = malloc(sizeof(struct node));
```

```
    n->data = x;
    n->next = NULL;
    n->prev = NULL;

    if (head == NULL) head = n;
    else {
        struct node *t = head;
        while (t->next) t = t->next;
        t->next = n;
        n->prev = t;
    }
}

void reverse() {
    struct node *t = head, *temp = NULL;

    while (t != NULL) {
        temp = t->prev;
        t->prev = t->next;
        t->next = temp;
        t = t->prev;
    }

    if (temp != NULL) head = temp->prev;
}

void display() {
    struct node *t = head;
    while (t) {
        printf("%d ", t->data);
        t = t->next;
    }
    printf("\n");
}

int main() {
    int n, x;
    printf("Enter number of nodes: ");
    scanf("%d", &n);

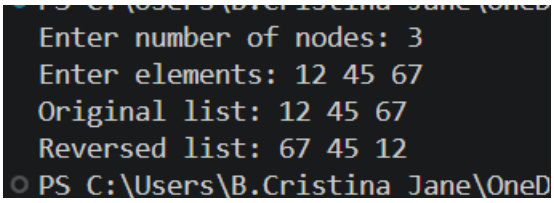
    printf("Enter elements: ");
```

```
    for (int i = 0; i < n; i++) {
      scanf("%d", &x);
      insert(x);
    }

    printf("Original list: ");
    display();
    reverse();
    printf("Reversed list: ");
    display();

    return 0;
}
```

OUTPUT:



5. **Write a C program to delete the middle element of a DLL.**

```
#include <stdio.h>
#include <stdlib.h>

struct node {
  int data;
  struct node *prev, *next;
};

struct node *head = NULL;

void insert(int x) {
  struct node *n = malloc(sizeof(struct node));
```

```
    n->data = x;
    n->next = NULL;
    n->prev = NULL;

    if (!head) head = n;
    else {
        struct node *t = head;
        while (t->next) t = t->next;
        t->next = n;
        n->prev = t;
    }
}

void deleteMiddle() {
    if (!head) {
        printf("List is empty\n");
        return;
    }

    struct node *slow = head, *fast = head;

    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
    }

    if (slow->prev) slow->prev->next = slow->next;
    else head = slow->next;
```

```
    if (slow->next) slow->next->prev = slow->prev;

    free(slow);
}

void display() {
    struct node *t = head;
    while (t) {
        printf("%d ", t->data);
        t = t->next;
    }
    printf("\n");
}

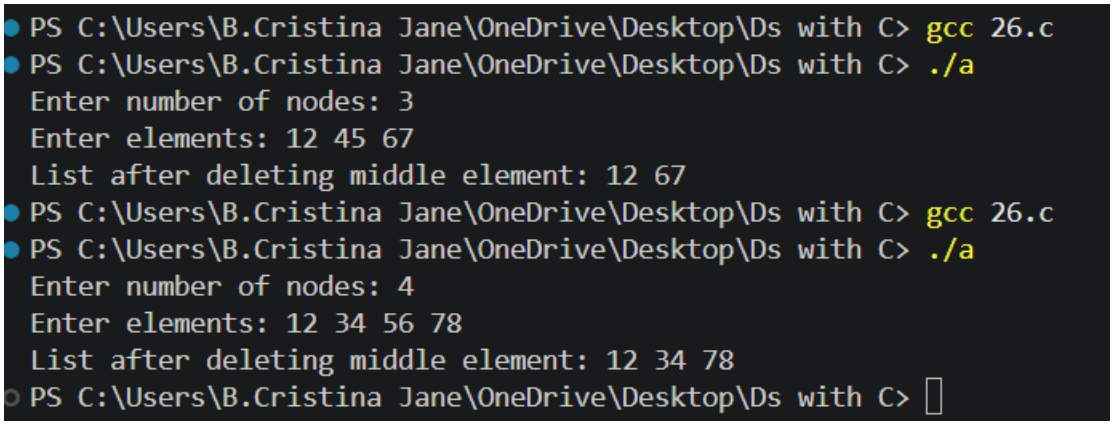
int main() {
    int n, x;
    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        insert(x);
    }

    deleteMiddle();
    printf("List after deleting middle element: ");
```

```
    display();
    return 0;
}
```

OUTPUT:



7.Remove duplicate elements from a (i) sorted DLL (ii) unsorted DLL.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *prev, *next;
};

struct node *head = NULL;

void insert(int x) {
   struct node *n = malloc(sizeof(struct node));
   n->data = x;
   n->next = NULL;
   n->prev = NULL;

   if (!head) head = n;
   else {
      struct node *t = head;
```

```
        while (t->next) t = t->next;
        t->next = n;
        n->prev = t;
    }
}

void removeDuplicates() {
    struct node *p = head;

    while (p) {
        struct node *q = p->next;
        while (q) {
            if (p->data == q->data) {
                struct node *del = q;
                q = q->next;

                if (del->prev) del->prev->next = del->next;
                if (del->next) del->next->prev = del->prev;
                free(del);
            } else {
                q = q->next;
            }
        }
        p = p->next;
    }
}

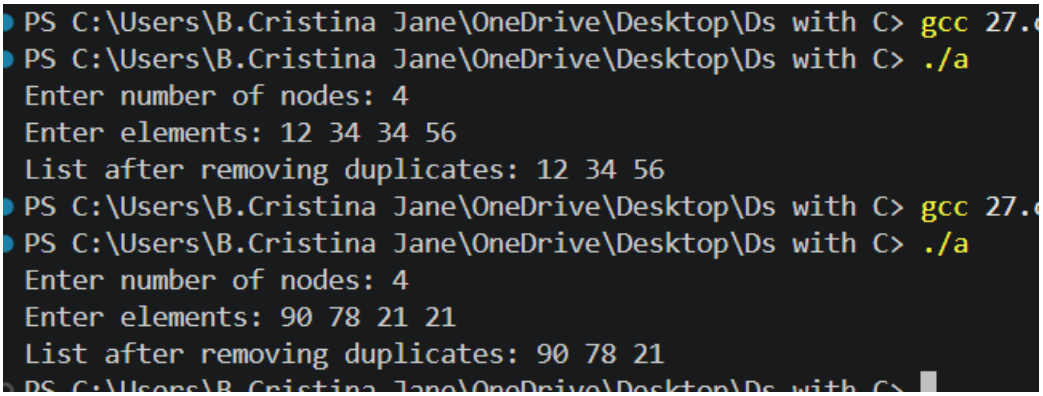
void display() {
    struct node *t = head;
    while (t) {
        printf("%d ", t->data);
        t = t->next;
    }
    printf("\n");
}

int main() {
    int n, x;
    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter elements: ");
```

```
    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        insert(x);
    }

    removeDuplicates();
    printf("List after removing duplicates: ");
    display();
    return 0;
}
```

OUTPUT:



8. Calculate the frequency count of a given integer in a circular DLL.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *prev, *next;
};

struct node *head = NULL;

void insert(int x) {
    struct node *n = malloc(sizeof(struct node));
    n->data = x;

    if (head == NULL) {
```

```
        head = n;
        n->next = n;
        n->prev = n;
    } else {
        struct node *last = head->prev;
        n->next = head;
        n->prev = last;
        last->next = n;
        head->prev = n;
    }
}

int main() {
    int n, x, key, count = 0;
    struct node *t;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

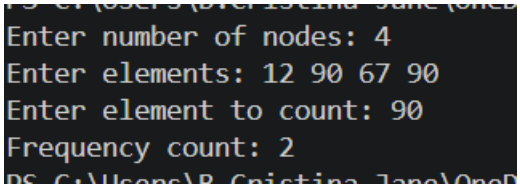
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) {
        scanf("%d", &x);
        insert(x);
    }

    printf("Enter element to count: ");
    scanf("%d", &key);

    if (head != NULL) {
        t = head;
        do {
            if (t->data == key) count++;
            t = t->next;
        } while (t != head);
    }

    printf("Frequency count: %d\n", count);
    return 0;
}
```

OUTPUT:



9. Search for a particular element in a binary tree.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *left, *right;
};

struct node *create(int a[], int i, int n) {
   if (i >= n) return NULL;

   struct node *root = malloc(sizeof(struct node));
   root->data = a[i];
   root->left = create(a, 2 * i + 1, n);
   root->right = create(a, 2 * i + 2, n);

   return root;
}

int search(struct node *root, int key) {
   if (root == NULL) return 0;
   if (root->data == key) return 1;

   return search(root->left, key) ||
          search(root->right, key);
}

int main() {
   int a[100], n, key;

   printf("Enter number of nodes: ");
   scanf("%d", &n);
```

```
    printf("Enter tree elements: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

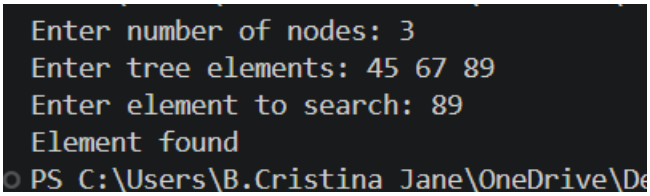
    struct node *root = create(a, 0, n);

    printf("Enter element to search: ");
    scanf("%d", &key);

    if (search(root, key))
        printf("Element found\n");
    else
        printf("Element not found\n");

    return 0;
}
```

OUTPUT:



10. Calculate the height or depth of a tree and print it.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left, *right;
};

struct node *create(int a[], int i, int n) {
    if (i >= n) return NULL;

    struct node *root = malloc(sizeof(struct node));
    root->data = a[i];
```

```
    root->left = create(a, 2 * i + 1, n);
    root->right = create(a, 2 * i + 2, n);

    return root;
}

int max(int a, int b) {
    return (a > b) ? a : b;
}

int height(struct node *root) {
    if (root == NULL) return 0;

    return 1 + max(height(root->left),
                   height(root->right));
}

int main() {
    int a[100], n;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

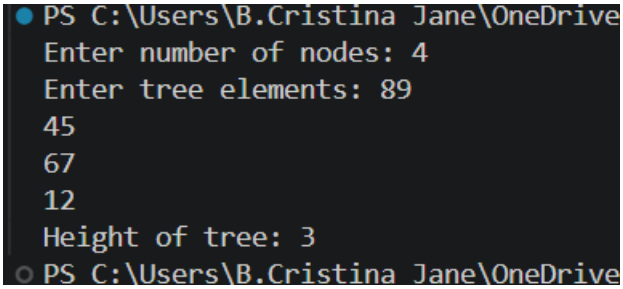
    printf("Enter tree elements: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    struct node *root = create(a, 0, n);

    printf("Height of tree: %d\n", height(root));

    return 0;
}
```

OUTPUT:



11.Given a binary tree, convert it to a doubly linked list keeping the same order (inorder traversal)

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *left, *right;
};

struct node *head = NULL, *prev = NULL;

struct node *create(int a[], int i, int n) {
   if (i >= n)
      return NULL;

   struct node *root = malloc(sizeof(struct node));
   root->data = a[i];
   root->left = create(a, 2 * i + 1, n);
   root->right = create(a, 2 * i + 2, n);

   return root;
}

void convert(struct node *root) {
   if (root == NULL)
      return;

   convert(root->left);

   if (prev == NULL)
```

```
        head = root;
    else {
        root->left = prev;
        prev->right = root;
    }

    prev = root;

    convert(root->right);
}

void display() {
    struct node *temp = head;

    printf("Doubly linked list: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->right;
    }
    printf("\n");
}

int main() {
    int a[100], n;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

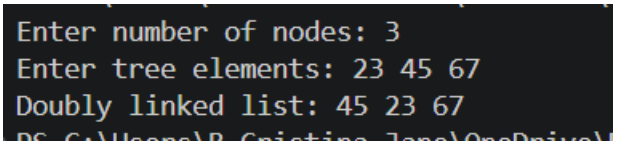
    printf("Enter tree elements: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    struct node *root = create(a, 0, n);

    convert(root);
    display();

    return 0;
}
```

OUTPUT:



12. Given the binary tree the task is to connect the nodes that are at the same level using next right pointers

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *left, *right, *nextRight;
};

struct node *create(int a[], int i, int n) {
   if (i >= n)
      return NULL;

   struct node *root = malloc(sizeof(struct node));
   root->data = a[i];
   root->left = create(a, 2 * i + 1, n);
   root->right = create(a, 2 * i + 2, n);
   root->nextRight = NULL;

   return root;
}

void connect(struct node *root) {
   struct node *queue[100];
   int front = 0, rear = 0;

   if (root == NULL)
      return;

   queue[rear++] = root;

   while (front < rear) {
      int count = rear - front;
      struct node *prev = NULL;
```

```
      for (int i = 0; i < count; i++) {
         struct node *cur = queue[front++];

         if (prev != NULL)
            prev->nextRight = cur;

         prev = cur;

         if (cur->left != NULL)
            queue[rear++] = cur->left;

         if (cur->right != NULL)
            queue[rear++] = cur->right;
      }

      prev->nextRight = NULL;
   }
}

void display(struct node *root) {
   struct node *queue[100];
   int front = 0, rear = 0, level = 1;

   if (root == NULL)
      return;

   queue[rear++] = root;

   while (front < rear) {
      int count = rear - front;

      printf("Level %d: ", level++);

      for (int i = 0; i < count; i++) {
         struct node *cur = queue[front++];

         printf("%d ", cur->data);

         if (cur->left != NULL)
            queue[rear++] = cur->left;
```

```
        if (cur->right != NULL)
            queue[rear++] = cur->right;
      }

      printf("\n");
   }
}

int main() {
   int a[100], n;

   printf("Enter number of nodes: ");
   scanf("%d", &n);

   printf("Enter tree elements: ");
   for (int i = 0; i < n; i++)
      scanf("%d", &a[i]);

   struct node *root = create(a, 0, n);

   connect(root);

   printf("Nodes connected at each level:\n");
   display(root);

   return 0;
}
```

OUTPUT:

```
Enter number of nodes: 2
Enter tree elements: 32 45
Nodes connected at each level:
Level 1: 32
Level 2: 45
PS C:\Users\B.Cristina Jane\OneDriv
```

13. Implement a priority queue using a binary search tree and perform the operation Insert.

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int priority;
   struct node *left, *right;
};

struct node *insert(struct node *root, int p) {
   if (root == NULL) {
      struct node *newnode =
         (struct node *)malloc(sizeof(struct node));

      newnode->priority = p;
      newnode->left = NULL;
      newnode->right = NULL;

      return newnode;
   }

   if (p < root->priority)
      root->left = insert(root->left, p);
   else if (p > root->priority)
      root->right = insert(root->right, p);
   else
      printf("Priority already exists\n");

   return root;
}

void display(struct node *root) {
   if (root != NULL) {
      display(root->left);
      printf("%d ", root->priority);
      display(root->right);
   }
}

int main() {
   struct node *root = NULL;
   int n, p;
```

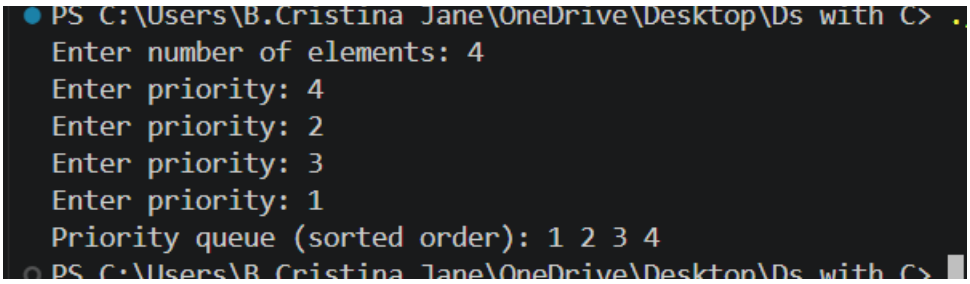
```
    printf("Enter number of elements: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter priority: ");
        scanf("%d", &p);
        root = insert(root, p);
    }

    printf("Priority queue (sorted order): ");
    display(root);
    printf("\n");

    return 0;
}
```

OUTPUT:



14. In a given binary search tree write a routines to delete a node with no children, one child and two children

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *left, *right;
};

struct node *insert(struct node *root, int x) {
    if (root == NULL) {
        struct node *newnode =
```

```
        (struct node *)malloc(sizeof(struct node));

    newnode->data = x;
    newnode->left = NULL;
    newnode->right = NULL;

    return newnode;
  }

  if (x < root->data)
    root->left = insert(root->left, x);
  else if (x > root->data)
    root->right = insert(root->right, x);

  return root;
}

struct node *findMin(struct node *root) {
  while (root->left != NULL)
    root = root->left;

  return root;
}

struct node *deleteNode(struct node *root, int key) {
  if (root == NULL)
    return NULL;

  if (key < root->data)
    root->left = deleteNode(root->left, key);

  else if (key > root->data)
    root->right = deleteNode(root->right, key);

  else {
    // Case 1: No children
    if (root->left == NULL && root->right == NULL) {
      free(root);
      return NULL;
    }
```

```
    // Case 2: One child
    else if (root->left == NULL) {
      struct node *temp = root->right;
      free(root);
      return temp;
    }

    else if (root->right == NULL) {
      struct node *temp = root->left;
      free(root);
      return temp;
    }

    // Case 3: Two children
    else {
      struct node *temp = findMin(root->right);
      root->data = temp->data;
      root->right = deleteNode(root->right, temp->data);
    }
  }

  return root;
}

void inorder(struct node *root) {
  if (root != NULL) {
    inorder(root->left);
    printf("%d ", root->data);
    inorder(root->right);
  }
}

int main() {
  struct node *root = NULL;
  int n, x, key;

  printf("Enter number of nodes: ");
  scanf("%d", &n);

  printf("Enter elements: ");
  for (int i = 0; i < n; i++) {
```

```
    scanf("%d", &x);
    root = insert(root, x);
  }

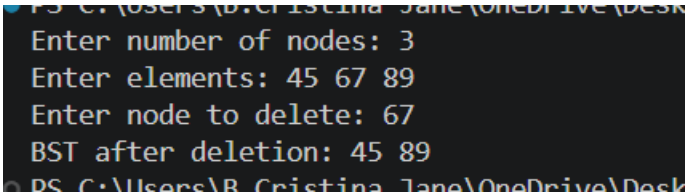
  printf("Enter node to delete: ");
  scanf("%d", &key);

  root = deleteNode(root, key);

  printf("BST after deletion: ");
  inorder(root);
  printf("\n");

  return 0;
}
```

OUTPUT:



15. Convert the given postfix expression to an expression tree

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

struct node {
  char data;
  struct node *left, *right;
};

struct node *stack[100];
int top = -1;

struct node *create(char c) {
  struct node *n = malloc(sizeof(struct node));
  n->data = c;
```

```
    n->left = n->right = NULL;
    return n;
}

int main() {
    char exp[100];

    printf("Enter postfix expression: ");
    scanf("%99s", exp);

    for (int i = 0; exp[i] != '\0'; i++) {
        char c = exp[i];
        struct node *n = create(c);

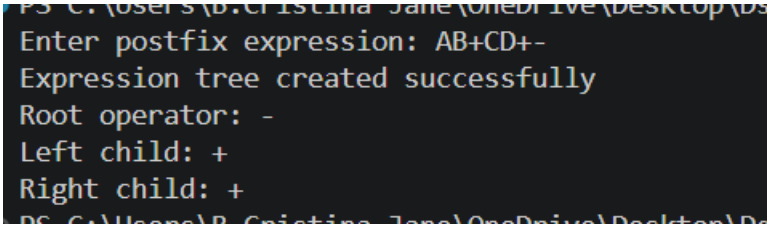
        if (isalnum((unsigned char)c)) {
            stack[++top] = n;
        } else {
            n->right = stack[top--];
            n->left = stack[top--];
            stack[++top] = n;
        }
    }

    struct node *root = stack[top];

    printf("Expression tree created successfully\n");
    printf("Root operator: %c\n", root->data);
    printf("Left child: %c\n", root->left->data);
    printf("Right child: %c\n", root->right->data);

    return 0;
}
```

OUTPUT:



16. Construct binary search tree using (i)inorder node representation,(ii) preorder and (iii)postorder

```
#include <stdio.h>
#include <stdlib.h>

struct node {
   int data;
   struct node *left, *right;
};

struct node *insert(struct node *root, int x) {
   if (root == NULL) {
      struct node *newnode =
         (struct node *)malloc(sizeof(struct node));

      newnode->data = x;
      newnode->left = NULL;
      newnode->right = NULL;

      return newnode;
   }

   if (x < root->data)
      root->left = insert(root->left, x);
   else if (x > root->data)
      root->right = insert(root->right, x);

   return root;
}

void inorder(struct node *root) {
   if (root != NULL) {
      inorder(root->left);
      printf("%d ", root->data);
      inorder(root->right);
   }
}

void preorder(struct node *root) {
```

```
    if (root != NULL) {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}

void postorder(struct node *root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

int main() {
    struct node *root = NULL;
    int n, x, i;

    printf("Enter number of nodes: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (i = 0; i < n; i++) {
        scanf("%d", &x);
        root = insert(root, x);
    }

    printf("Inorder traversal: ");
    inorder(root);

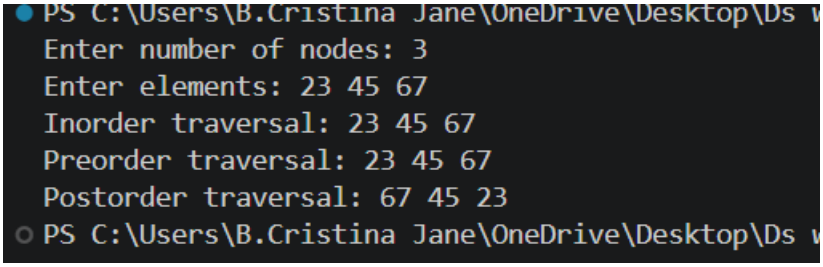
    printf("\nPreorder traversal: ");
    preorder(root);

    printf("\nPostorder traversal: ");
    postorder(root);

    printf("\n");

    return 0;
}
```

OUTPUT:



17. Implement heap sort for a set of numbers

```
#include <stdio.h>

void heapify(int a[], int n, int i) {
   int largest = i;
   int left = 2 * i + 1;
   int right = 2 * i + 2;
   int temp;

   if (left < n && a[left] > a[largest])
      largest = left;

   if (right < n && a[right] > a[largest])
      largest = right;

   if (largest != i) {
      temp = a[i];
      a[i] = a[largest];
      a[largest] = temp;

      heapify(a, n, largest);
   }
}

void heapSort(int a[], int n) {
   int temp;

   for (int i = n / 2 - 1; i >= 0; i--)
      heapify(a, n, i);
```

```
    for (int i = n - 1; i > 0; i--) {
        temp = a[0];
        a[0] = a[i];
        a[i] = temp;

        heapify(a, i, 0);
    }
}

int main() {
    int a[100], n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements: ");
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    heapSort(a, n);

    printf("Sorted elements: ");
    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
    return 0;
}
```

OUTPUT:

```
Enter number of elements: 3
Enter elements: 90
12
45
Sorted elements: 12 45 90
```